WEB TECHNOLOGIES

Unit 3: JavaScript & Client-Side Scripting

Rgpvnotes.tech - Your Academic Excellence Partner

UNIT 3: Client-Side Scripting | What is JavaScript | Simple JavaScript | Variables | Functions | Conditions | Loops and Repetition

Table of Contents

Topic 1: Client-Side Scripting & What is JavaScript

Topic 2: Simple JavaScript (Syntax & Inclusion)

Topic 3: Variables and Data Types

Topic 4: Functions in JavaScript

Topic 5: Conditional Statements (if, else, switch)

Topic 6: Loops and Repetition (for, while, do-while)

Revision Notes

Important Questions

1 Client-Side Scripting & What is JavaScript

Definition:

Client-side scripting refers to executing scripts (code) directly within the user's web browser, rather than on the web server. **JavaScript (JS)** is a lightweight, interpreted, object-oriented programming language primarily used as the standard client-side scripting language for the web.

Detailed Explanation:

In web development, there are two main environments where code can run: the Server (backend) and the Client (frontend/browser). Client-side scripting involves sending the raw source code to the user's browser, which then translates and executes it locally. This approach drastically reduces server load and provides immediate feedback to users without needing to reload the webpage.

JavaScript is the undisputed king of client-side scripting. Created by Brendan Eich in 1995, JS transforms static HTML pages into dynamic, interactive experiences. While HTML provides the structure and CSS provides the styling, JavaScript dictates the behavior (e.g., what happens when a button is clicked, validating a form before submission, creating image sliders, or updating data asynchronously via AJAX). JavaScript is an *interpreted* language, meaning it doesn't need to be compiled into machine code before execution; the browser's JS Engine (like Google Chrome's V8) reads and executes it on the fly. It is also *dynamically typed* and heavily relies on the **DOM (Document Object Model)** to interact with and modify HTML elements dynamically.

🔍 Key Features of JavaScript:

- ✓ **Interpreted Language:** Executed line-by-line by the browser.
- ✓ **Event-Driven:** Code executes in response to user actions (clicks, key presses, mouse movements).
- ✓ **Object-Oriented (Prototype-based):** Supports objects, methods, and properties.
- ✓ **Platform Independent:** Runs on any operating system that has a modern web browser.
- ✓ **DOM Manipulation:** Can dynamically add, delete, or change HTML elements and CSS styles.

📊 Client-Side Scripting Flow

```
[ WEB SERVER ] ------(Sends HTML + CSS + JS)-----> [ USER'S BROWSER ]
                                                          |
                                                (Browser parses HTML/CSS)
                                                          |
                                              (Browser's JS Engine runs JS)
                                                          |
                                            [ Dynamic/Interactive Webpage ]
```

✅ Advantages of Client-Side Scripting (JS):

- ✓ **Speed:** Immediate response to user input (no waiting for the server).
- ✓ **Reduced Server Load:** Computations happen on the user's device.
- ✓ **Rich Interfaces:** Enables drag-and-drop, sliders, and complex UI components.

❌ Disadvantages:

- ✗ **Security Risks:** Code is visible to the user; malicious scripts (XSS) can be a threat.
- ✗ **Browser Dependency:** Different browsers might interpret old JS slightly differently.
- ✗ **Can be Disabled:** Users can turn off JS in browser settings, breaking functionality.

🌍 Applications of JavaScript:

- Form validation (checking if email format is correct before sending).
- Creating interactive maps (like Google Maps).
- Developing web-based games and animations.

- Building Single Page Applications (SPAs) using frameworks like React or Angular.

Hinglish Explanation:
Client-side scripting *ka matlab hai ki code server pe nahi, balki user ke apne browser (jaise Chrome) mein run hota hai.* ***JavaScript (JS)*** *ek aisi programming language hai jo websites ko "zinda" (interactive) banati hai. HTML se dhancha banta hai, CSS se design, aur JS se usme actions aate hain (jaise button click karne pe popup aana). Ye fast hoti hai kyunki page reload kiye bina hi kaam kar deti hai.*

2 Simple JavaScript (Syntax & Inclusion)

Definition:

Simple JavaScript involves the basic syntax, rules, and methods used to write and integrate JS code into an HTML document. It covers how statements are written, how comments are added, and how data is outputted to the user.

Detailed Explanation:

To use JavaScript in a web page, the code must be enclosed within `<script>` and `</script>` tags. These tags can be placed in the `<head>` or the `<body>` of the HTML document. However, placing scripts at the bottom of the `<body>` is recommended to ensure the HTML loads before the JS tries to interact with it.
JavaScript can be included in three ways:

1. **Inline:** Placed directly inside HTML element attributes (e.g., `onclick="alert('Hi')"`).
2. **Internal:** Written inside `<script>` tags within the same HTML file.
3. **External:** Written in a separate `.js` file and linked using `<script src="script.js"></script>`. (Best Practice for reusability and clean code).

JavaScript Output Methods: Unlike other languages, JS doesn't have a single standard "print" function. It provides multiple ways to display output depending on the requirement:

- `console.log()`: Prints output to the browser's developer console (used for debugging).
- `alert()`: Displays a popup alert box to the user.
- `document.write()`: Writes directly to the HTML document (rarely used today as it overwrites the page).
- `innerHTML`: Modifies the content of an existing HTML element.

Example: Simple JavaScript Implementation

```html
<!DOCTYPE html>
<html>
<body>

    <h2>My First JavaScript</h2>
    <p id="demo">This text will change.</p>

    <button onclick="changeText()">Click Me!</button>
```

```
    <script>
        // This is an internal JavaScript function
        function changeText() {
            // Using innerHTML to change content
            document.getElementById("demo").innerHTML = "Hello World!";

            // Outputting to console for debugging
            console.log("Button was clicked!");
        }
    </script>

</body>
</html>
```

Key Rules of JS Syntax:

- ✓ **Case Sensitivity:** `Variable` and `variable` are treated as different.
- ✓ **Statements:** End with a semicolon (`;`) (Highly recommended though optional).
- ✓ **Comments:** Single line uses `//`, multi-line uses `/* ... */`.

Hinglish Explanation:

JS ko HTML me use karne ke liye `<script>` tag ka use hota hai. Ise hum HTML ke andar bhi likh sakte hain (Internal) ya alag file banakar link kar sakte hain (External - jo ki best tareeka hai). Output dekhne ke liye hum `alert()` (popup ke liye), `console.log()` (developer tab ke liye), ya `innerHTML` (page ka text badalne ke liye) use karte hain.

3 Variables and Data Types

Definition:

Variables are named containers used to store data values in memory. **Data Types** specify the kind of data that can be stored and manipulated within a program.

Detailed Explanation:

In JavaScript, variables can be declared using three keywords: `var`, `let`, and `const`.

- **var:** The old way of declaring variables. It has global or function-level scope and allows redeclaration, which can lead to bugs.
- **let:** Introduced in ES6 (modern JS). It has block-level scope (confined within `{}`) and cannot be redeclared in the same scope. Recommended for variables whose values will change.
- **const:** Also block-scoped. Used for variables whose values must **not** change (constants) after initial assignment.

JavaScript Data Types: JS is loosely (dynamically) typed, meaning a variable can hold a string, and later

be reassigned to a number without errors. Data types are categorized into two groups:

1. Primitive Data Types: Hold a single value.

- `String`: Text, enclosed in quotes (e.g., `"Hello"`, `'Web'`).
- `Number`: Integers and decimals (e.g., `25`, `99.99`).
- `Boolean`: Logical entities having only two values: `true` or `false`.
- `Undefined`: A variable that has been declared but not assigned a value.
- `Null`: Represents the intentional absence of any object value.

2. Non-Primitive (Reference) Data Types: Can hold collections of values.

- `Object`: A collection of key-value pairs (e.g., `{name: "John", age: 30}`).
- `Array`: An ordered list of values enclosed in square brackets (e.g., `[10, 20, 30]`).

💻 Example: Variables and Data Types

```
let studentName = "Rahul";        // String
let age = 20;                     // Number
let isPassed = true;              // Boolean
let marks;                        // Undefined

const COLLEGE_NAME = "RGPV";      // Constant (Cannot be changed)

// Array (List of items)
let subjects = ["HTML", "CSS", "JS"];

// Object (Key-value pairs)
let studentProfile = {
    firstName: "Rahul",
    enrollment: "0101CS20"
};

console.log(studentName + " studies at " + COLLEGE_NAME);
```

🔑 Key Rules for Naming Variables:

✓ Names must begin with a letter, `$`, or `_`.

✓ Names cannot start with a number.

✓ Reserved keywords (like `function`, `return`, `let`) cannot be used as variable names.

✓ CamelCase is standard convention (e.g., `myFirstName`).

👉 *Hinglish Explanation:*

Variables *container ki tarah hote hain jisme data store hota hai. Modern JS me hum* `let` *(jiski value badal sakti hai) aur* `const` *(jiski value fix rehti hai) ka use karte hain.* ***Data Types*** *batate hain ki data kaisa hai: text hai to* `String`*, number hai to* `Number`*, sahi/galat hai to* `Boolean`*. JS dynamically typed hai, iska matlab aapko batana nahi padta ki type kya hai, browser khud samajh jata hai.*

4 Functions in JavaScript

Definition:

A **JavaScript Function** is a reusable block of code designed to perform a particular task. It is executed only when "called" or "invoked" by an event, another part of the script, or manually.

Detailed Explanation:

Functions are the building blocks of JavaScript. They allow developers to write a piece of code once and use it multiple times without rewriting it. This promotes the **DRY (Don't Repeat Yourself)** principle, making code modular, cleaner, and easier to maintain.

A function is defined using the `function` keyword, followed by a name, parentheses `()`, and curly brackets `{}`. The code to be executed is placed inside the curly brackets.

Parameters and Arguments: Functions can accept inputs called *parameters*. When the function is invoked, the actual values passed to it are called *arguments*.

Return Statement: A function can perform calculations and send a result back to the place where it was called using the `return` keyword. When JS reaches a return statement, the function stops executing.

Example: Declaring and Calling a Function

```
// 1. Function Declaration with Parameters
function calculateArea(length, width) {
    let area = length * width;
    return area; // Returns the result
}

// 2. Function Invocation (Calling)
let rectArea = calculateArea(10, 5);

// 3. Output
console.log("The area of the rectangle is: " + rectArea);
```

Advantages of Functions:

✓ **Code Reusability:** Write once, use multiple times.

✓ **Modularity:** Breaks down complex problems into smaller, manageable chunks.

✓ **Readability:** Makes code easier to understand and debug.

Hinglish Explanation:

Functions *ek machine ki tarah kaam karte hain. Aap ek baar code ka block banate hain, aur jab bhi us kaam ki zaroorat ho, bas us function ka naam likh kar use "call" kar lete hain. Isse hume same code baar-baar nahi likhna padta. Functions me hum input (parameters) bhej sakte hain aur wo hume output (return) de sakta hai.*

5 Conditional Statements

Definition:

Conditional Statements are control structures used to perform different actions based on different conditions. They allow the program to make decisions and execute specific blocks of code only if a specified condition evaluates to `true`.

Detailed Explanation:

In real-world applications, programs need to make decisions. For example, if a user's password is correct, log them in; otherwise, show an error. JavaScript provides several conditional statements to handle such logic:

1. The `if` Statement: Executes a block of code if a specified condition is true.

2. The `else` Statement: Used alongside `if`. It executes a block of code if the same condition is false.

3. The `else if` Statement: Specifies a new condition to test if the first condition is false. Used when checking multiple conditions.

4. The `switch` Statement: Used to perform different actions based on different specific conditions. It is an alternative to using multiple `else if` statements and is often cleaner when checking a single variable against many exact values. It uses the `break` keyword to stop execution once a match is found.

Example 1: if, else if, else

```
let marks = 75;

if (marks >= 80) {
    console.log("Grade: A");
} else if (marks >= 60) {
    console.log("Grade: B"); // This block will execute
} else {
    console.log("Grade: Fail");
}
```

Example 2: switch Statement

```
let day = 3;
let dayName;

switch (day) {
    case 1:
        dayName = "Monday";
        break;
    case 2:
        dayName = "Tuesday";
        break;
    case 3:
        dayName = "Wednesday"; // Matches here
        break;
    default:
```

```
        dayName = "Invalid Day";
}
console.log(dayName);
```

Key Points:

- ✓ Conditions are evaluated using comparison operators (`==`, `===`, `>`, `<`, etc.).
- ✓ Always use `===` (strict equality) over `==` as it checks both value and data type.
- ✓ In a `switch` statement, if you forget the `break` keyword, the code will continue to execute the next cases (known as "fall-through").

Hinglish Explanation:

***Conditions** program me decision lene ke kaam aati hain. Agar condition sahi (true) hai, toh `if` ka code run hoga, varna `else` ka. Agar bohot saari conditions check karni ho toh `else if` ya `switch` ka use karte hain. Jaise: "Agar umar 18 se upar hai toh vote do, warna nahi do."*

6 Loops and Repetition

Definition:

Loops are control structures used to repeatedly execute a block of code as long as a specified condition evaluates to `true`. They save time, reduce code size, and are essential for working with arrays and repetitive tasks.

Detailed Explanation:

If you need to print numbers from 1 to 100, writing `console.log()` a hundred times is highly inefficient. Loops automate this repetition. JavaScript supports several types of loops:

1. The `for` Loop: Used when the number of iterations (how many times the loop should run) is known beforehand. It contains three expressions separated by semicolons: Initialization (start point), Condition (when to stop), and Increment/Decrement (step size).

2. The `while` Loop: Used when the number of iterations is unknown. It repeats a block of code *while* a specified condition is true. The condition is checked *before* the code executes.

3. The `do...while` Loop: A variant of the while loop. It executes the code block once *before* checking the condition, and then repeats the loop as long as the condition is true. It guarantees at least one execution.

Loop Control Statements:

- `break`: Instantly stops and jumps out of the loop.
- `continue`: Skips the current iteration and jumps to the next iteration of the loop.

Example 1: The 'for' Loop

```
// Syntax: for (initialization; condition; increment)
for (let i = 1; i <= 5; i++) {
    console.log("Iteration number: " + i);
}
// Output will print lines 1 to 5
```

💻 Example 2: The 'while' Loop

```
let count = 1;

while (count <= 3) {
    console.log("Count is " + count);
    count++; // Crucial: Prevents infinite loop
}
```

✅ Advantages of Loops:

✓ **Efficiency:** Executes repetitive tasks quickly with minimal code.

✓ **Array Traversal:** Essential for iterating through lists of data (e.g., displaying a list of products).

❌ Disadvantages/Risks:

✗ **Infinite Loops:** If the stopping condition is never met (e.g., forgetting to increment the counter), the loop will run forever and crash the user's browser.

*👉 **Hinglish Explanation:***
***Loops** ka use ek hi kaam ko baar-baar karwane ke liye hota hai. Agar humein pata hai ki loop kitni baar chalana hai (jaise 10 baar), toh hum `for` loop use karte hain. Agar nahi pata aur kisi condition pe depend karta hai, toh `while` loop use karte hain. Hamesha dhyan rakhein ki condition aisi ho jo kabhi na kabhi false ho jaye, varna loop infinite ho jayega aur browser hang ho jayega.*

🔄 QUICK REVISION NOTES

1. JavaScript & Client-Side:
JS is a lightweight, interpreted language that runs on the user's browser. It manipulates HTML/CSS dynamically, creating interactive websites without server reloads.

2. Syntax & Inclusion:
Written inside `<script>` tags. Can be inline, internal, or external (.js file). Output using `console.log()`, `alert()`, or `innerHTML`.

3. Variables & Data Types:
Declare using `var`, `let`, `const`. Primitive types: String, Number, Boolean, Null, Undefined. Non-Primitive:

Objects, Arrays.

4. Functions:
Reusable blocks of code. Declared with `function` keyword. Can take inputs (parameters) and return outputs. Promotes DRY principle.

5. Conditions:
Make decisions using `if`, `else if`, `else`, and `switch` based on boolean conditions (true/false).

6. Loops:
Repeat code efficiently. `for` loop (known iterations), `while` loop (condition-based), `do-while` (executes at least once).

IMPORTANT QUESTIONS (6 Marks Each)

Q1. What do you understand by Client-Side Scripting? Explain the advantages and limitations of using JavaScript for client-side operations. 6 Marks

Expected Answer Points:

- Definition of Client-Side Scripting (executes on browser, not server).
- Role of JavaScript (adds interactivity, DOM manipulation).
- Advantages: Faster response, reduced server load, rich UI.
- Limitations: Security risks (code is visible), browser dependencies, can be disabled by users.

Q2. Explain the different ways to include JavaScript in an HTML document. Which method is considered the best practice and why? 6 Marks

Expected Answer Points:

- Inline JS (inside HTML tags using attributes like onclick).
- Internal JS (inside `<script>` tags in head or body).
- External JS (linking a .js file using the src attribute).
- Best practice is External because it separates content (HTML) from behavior (JS), makes code reusable, and allows browser caching.

Q3. What are Variables in JavaScript? Explain the differences between var, let, and const with suitable examples. 6 Marks

Expected Answer Points:

- Definition of variables as data containers.

- `var`: Function/global scope, can be redeclared (older way).
- `let`: Block scope, cannot be redeclared, value can be updated (modern way).
- `const`: Block scope, cannot be redeclared, value cannot be reassigned (constant).
- Provide brief code syntax for each.

Q4. Define Functions in JavaScript. How are parameters and return values used in a function? Provide a code example. 6 Marks

Expected Answer Points:

- Definition: Reusable block of code for a specific task.
- Explain parameters (inputs received by function) and arguments (values passed).
- Explain the `return` keyword (sends output back to caller).
- Code example: A simple function like add(a, b) returning a+b.

Q5. Discuss Loop control structures in JavaScript. Differentiate between 'for' and 'while' loops giving the syntax for both. 6 Marks

Expected Answer Points:

- Definition of Loops (repeating code block based on condition).
- `for` loop: Used when iterations are known. Syntax includes initialization, condition, increment.
- `while` loop: Used when iterations are unknown, checks condition before running.
- Explain the risk of infinite loops if condition never becomes false.

These notes are comprehensive and exam-ready

Practice the code examples provided to secure full marks in programming questions!

© 2026 - Your Academic Excellence Partner

These notes are created for educational purposes to help students master Web Technologies.